

IoT-Based Smart Parking System

Using NodeMCU ESP8266, IR Sensors, Servo Motor,
LCD Display, and MQTT Protocol

Project Report

23BCT0009 – Garv Katariya R

23BCT0010 – Kshitij Mishra

23BCT0036 – Jothi Baskar

Abstract

The issue of urban parking has evolved into an urgent problem due to the fact that the number of vehicles is ever increasing in cities across the globe. Drivers often find it hard to find empty parking spaces within a short period of time leading to fuel wastage, wastage of time and congestion at parking facility entrances. Traditional parking systems are based on manual monitoring, which is not scalable and efficient.

This report will cover the design and implementation of an IoT-Based Smart Parking System which involves three IR sensors that perform different tasks, a servo-controlled gate, an I2C LCD display, and MQTT-based cloud communication which will provide real-time monitoring of parking slots and automated control of the gate. The microcontroller, Wi-Fi communication, and central processing unit is the NodeMCU ESP8266. The parking area has two permanently mounted IR sensors, one of which is attached to each parking slot, to monitor the parking occupancy at all times. The third IR sensor will be placed in the entry gate to sense an approaching motor vehicle.

At the entry point, when the gate sensor notices the presence of a vehicle, the system will check the occupancy of the two parking slots. When one of the slots is free, the servo motor turns to mimic an open gate and the vehicle is allowed inside. When the two slots are taken up the gate will be closed. The occupancy status of the live slots on-site is displayed on an I2C LCD display, whereas a mobile dashboard connected to MQTT allows one to monitor the occupancy status remotely. This architecture provides a low-cost, responsive and practical prototype that can be used in smart parking and smart city implementations.

1. Introduction

1.1 Motivation

The rapid urbanisation has resulted in the significant growth in the number of private cars, and this has been putting tremendous pressure on the urban parking system. In the majority of cities, the parking lots do not have a real-time status update system and drivers have to search the parking lots manually to find free spaces. This causes unnecessarily driving in the parking lot, more carbon emissions and frustration to the drivers.

The Internet of Things (IoT) offers a chance to cope with this issue at a reasonable price. The infrared sensors, as well as wireless communication systems, based on MQTT and low-cost microcontrollers like the NodeMCU ESP8266, enable real-time information capture, processing, and transmission without costly dedicated hardware. One of the design considerations of this project would be to make the entry gate smart enough, only to open when a vehicle is physically at the gate and a free slot exists in the interior to prevent the gate from making unnecessary movements.

Educational objectives also drive the project - to bring together the knowledge of embedded systems, sensor networks, wireless communication protocols, actuator control and human-machine interfaces into one cohesive IoT application.

1.2 Scope of the Project

The areas covered in this project include:

Design and construction of a 2-slot parking system based on specific IR sensors at each slot.

- Separate gate-entry IR sensor that will trigger gate evaluation only when a vehicle is detected.
- Local depiction of parking status on an I2C LCD display in real-time.
- Automated gate control via a servo motor - gate is only opened on detection of a vehicle and slot is vacant.
- MQTT publishing of data over Wi-Fi through the ESP8266 module of the NodeMCU.

Remote parking data monitoring through the IoT MQTT Panel mobile app.

Scalability and modularity of the proposed system architecture.

2. Problem Formulation

2.1 Problem Statement

In urban and semi-urban situations, there is a tendency to use parking spaces inefficiently because there is no real-time occupancy information. Drivers are forced to move physically through parking areas to find an empty parking slot, a factor that contributes to traffic jam at the entrance areas, fuel wastage, pollution and waiting. Manual attendants are costly, inaccurate, and unable to support the expanding cities.

Another issue is at the gate level: without some sort of vehicle-detection system at the entry point, a gate system would either have to open on a timer (inefficient) or it would have to be operated manually (renders the entire automation pointless). This project will solve both issues by inquiring: How can I implement a low-cost embedded IoT system to detect occupancy of parking slots, check the presence of a vehicle at the gate, conditionally control the access of the gate and broadcast parking availability locally and remotely, without using costly proprietary hardware?

2.2 Software Tools Used

Software / Platform	Purpose
Arduino IDE	Writing, compiling, uploading C++ firmware to NodeMCU ESP8266
MQTT Broker (broker.emqx.io)	Free public MQTT broker for routing parking data messages between the device and the mobile app
IoT MQTT Panel (Android/iOS App)	Mobile application that subscribes to MQTT topics and displays real-time slot and gate status on a dashboard
ESP8266WiFi Library	Manages Wi-Fi connectivity of the NodeMCU to a local hotspot or router
PubSubClient Library	Enables MQTT publish and subscribe functionality on the ESP8266 platform

LiquidCrystal_I2C Library	Drives I2C LCD display for local human-readable output of parking status
ESP8266Servo Library	Controls the servo motor angle for gate open/close simulation on ESP8266

2.3 Hardware Components Used

Component	Quantity	Role in System
NodeMCU ESP8266	1	Central microcontroller and Wi-Fi communication unit
IR Obstacle Sensor (Slot)	2	Monitors occupancy of Slot 1 and Slot 2 continuously
IR Obstacle Sensor (Gate)	1	Detects vehicle presence at the parking entry gate
Servo Motor (SG90 or similar)	1	Simulates the automated entry gate — opens only when a vehicle is detected AND a slot is free
I2C LCD Display (16x2)	1	Displays slot occupancy status and free slot count locally at the parking entry
Breadboard	1	Prototyping and connection platform
Jumper Wires	As required	Electrical connections between all components
USB Cable (Micro-USB)	1	Power supply and serial programming interface
5V External Power Supply	1 (optional)	Stable power source for servo motor to prevent brownout on the NodeMCU

2.4 Pin Configuration / Hardware Connections

IR Sensors

Sensor	Role	OUT Pin	NodeMCU Pin
IR Sensor 1	Slot 1 occupancy	OUT	D5
IR Sensor 2	Slot 2 occupancy	OUT	D6
IR Sensor 3 (Gate)	Vehicle at gate detection	OUT	D7

All IR Sensors	Power	VCC / GND	3.3V / GND
----------------	-------	-----------	------------

I2C LCD Display

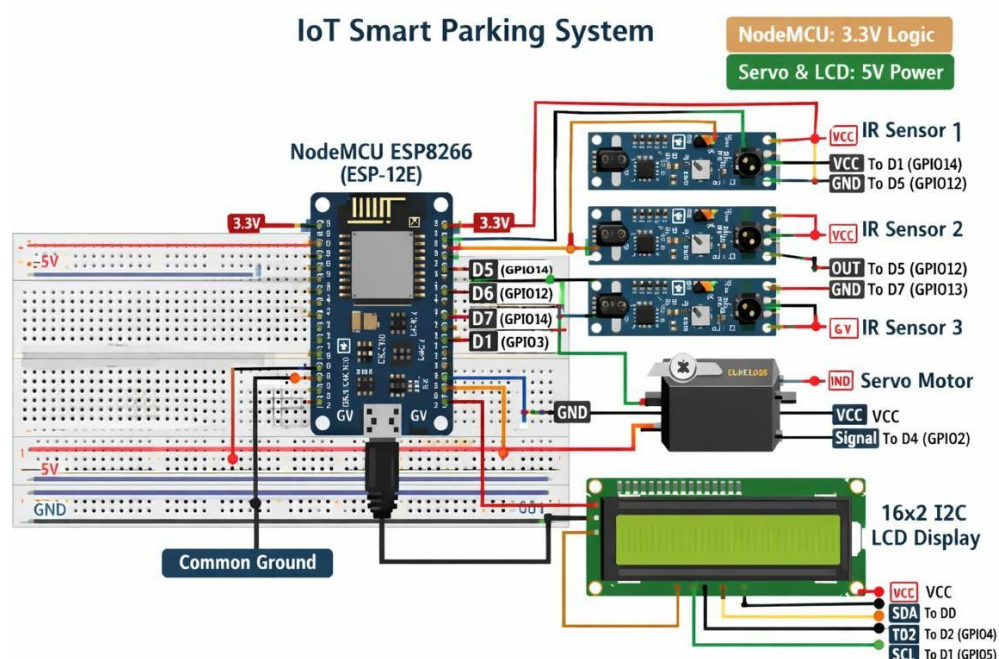
LCD Pin	Connection	NodeMCU Pin
SDA	I2C Data Line	D2
SCL	I2C Clock Line	D1
VCC / GND	Power	3.3V or 5V / GND

Servo Motor

Servo Pin	Connection	NodeMCU Pin
Signal (Orange)	PWM Control	D4
VCC (Red)	Power	5V (external recommended)
GND (Brown/Black)	Ground	GND (common)

3. Implementation

3.1 Circuit Diagram



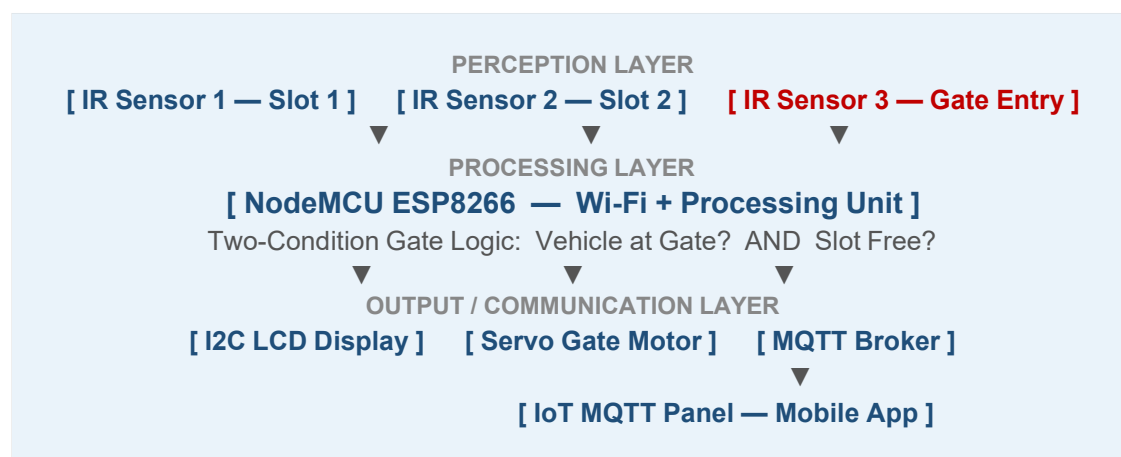
3.2 System Architecture Overview

The network is structured to be a layered IoT architecture. At the perception layer, two slot IR sensors constantly observe the physical occupancy of the two parking spaces and a special gate IR sensor observes the entrance point of an approaching vehicle. On the processing layer, the NodeMCU ESP8266 reads all three sensor inputs, assesses a conditional gate-open logic and actuators are driven. The I2C LCD will display on-site parking status at the display layer. MQTT, at the communication layer, publishes parking information through Wi-Fi to cloud broker. Lastly, the IoT MQTT Panel mobile app uses and displays the data at the application layer remotely.

The most important design difference between this system and a simple gate-timer solution is that the gate logic has two conditions: the gate servo is activated only when (1) the vehicle at the entry is detected by the gate IR sensor, and (2) when at least one of two parking slots is free. This will avoid unnecessary use of gates and unnecessary admission of vehicles to a complete lot.

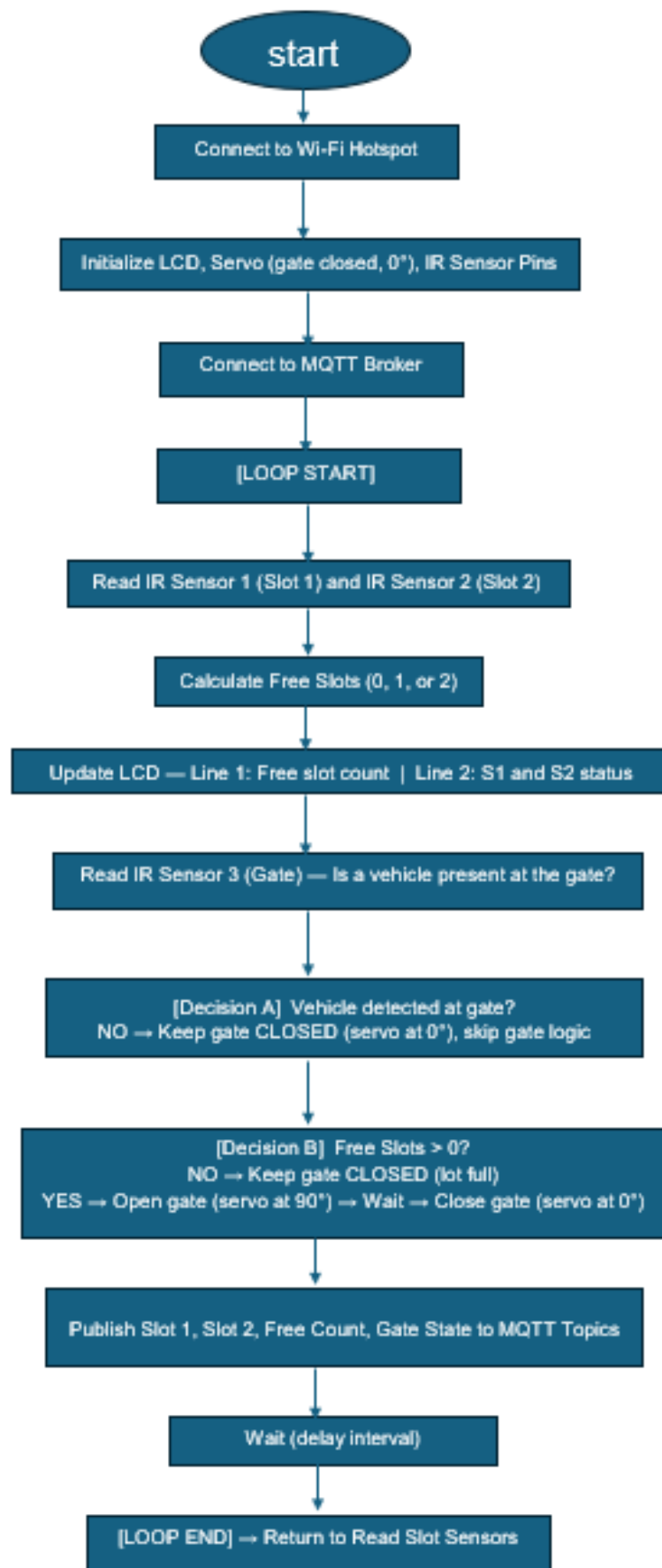
3.3 Block Diagram

The following block diagram illustrates the high-level architecture of the system:



3.4 System Flowchart

The operational logic of the firmware is described through the following flowchart:



3.5 Working Principle

When the NodeMCU ESP8266 is powered, all the peripherals are initialised. The servo motor is ordered to 0-degree, which is the closed gate. I2C LCD is started and a welcome message shown. IR sensors are set up with three digital input pins. The firmware then tries to connect to the configured Wi-Fi network with the stored SSID and password.

When Wi-Fi connectivity is provided, the firmware uses a unique client identifier to connect to the public MQTT broker (such as broker.hivemq.com on port 1883) via Wi-Fi. This enables the system to send status messages to pre-determined topic strings to which the mobile application is subscribing.

The firmware reads the two slot sensors in the main operational loop. IR Sensor 1 is watching Slot 1 and IR Sensor 2 is watching Slot 2. In the case of a parked car that is covering a sensor, the sensor's output pin goes LOW (object detected), and the slot is marked as occupied. When the slot is not occupied, the pin will be HIGH, and the slot is indicated as free. Using these two readings, the system identifies the number of available slots (0, 1 or 2). The LCD is instantly updated to show the number of free slots and the status of each slot individually.

The IR Sensor 3, which is placed on the parking entrance gate, is then read by the firmware. The only aim of this sensor is to determine the presence and wait of a vehicle at the gate. In case no vehicle is present, the gate logic is completely bypassed and the servo is left in the closed position - this is a key efficiency consideration that does not allow the gate to consider or take action without reason.

In case the gate sensor recognizes a vehicle, the system will then consider the availability of slots. When there is at least one slot available, the servo motor moves the gate open by rotating to 90 degrees. The servo is brought back to 0 degrees after a specified waiting time (to allow the vehicle to pass by the gate) to open the gate. When both slots are occupied when the vehicle reaches the gate the gate is left closed and no servo motion is applied. An indication of the full lot can be displayed on the LCD as well.

In every loop, the NodeMCU broadcasts the state of both slots, the number of free slots and the state of the gate in their respective MQTT topics. These messages are relayed to the broker and passed on to the IoT MQTT Panel application, which is set with the same topic subscriptions on his or her mobile device, and displays the dashboard in real-time.

3.6 MQTT Topic Structure

MQTT Topic	Description
smartparking/s1	Occupancy state of Slot 1 (Occupied / Free)
smartparking/s2	Occupancy state of Slot 2 (Occupied / Free)
smartparking/free	Number of currently free parking slots (0, 1, or 2)
smartparking/gate	Gate status (Open or Closed) driven by two-condition logic

Note: The state of the gate IR sensor is not meant to be published as a separate MQTT topic but is internally utilized by the firmware to make a decision and can be published as a diagnostic aid.

3.7 LCD Display Output

```
Free: 1 / 2
```

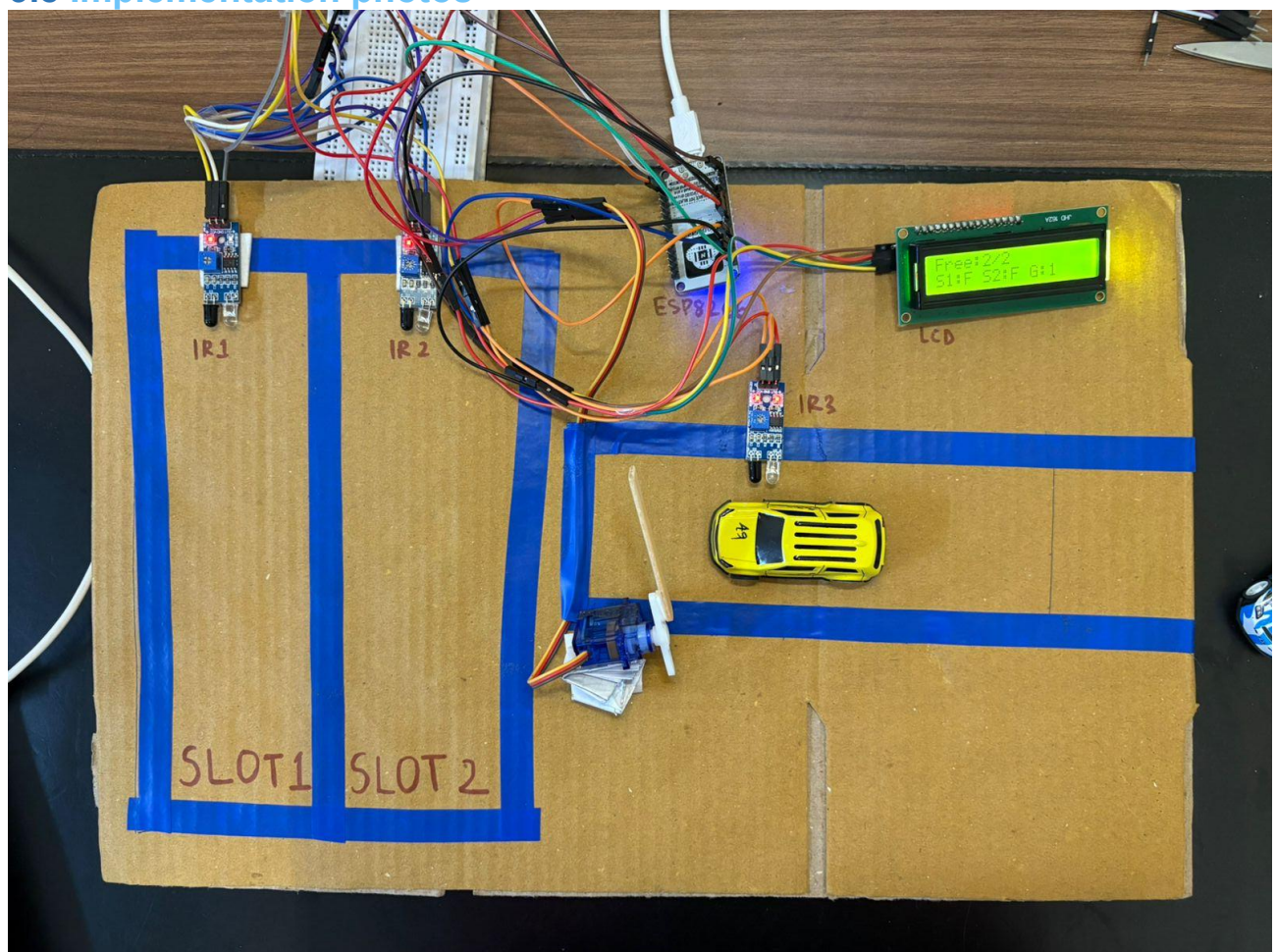
The I2C LCD display is connected to the NodeMCU via the I2C protocol using SDA and SCL alone. An example display output will be as below in a situation where Slot 1 is in use and Slot 2 is not in use:

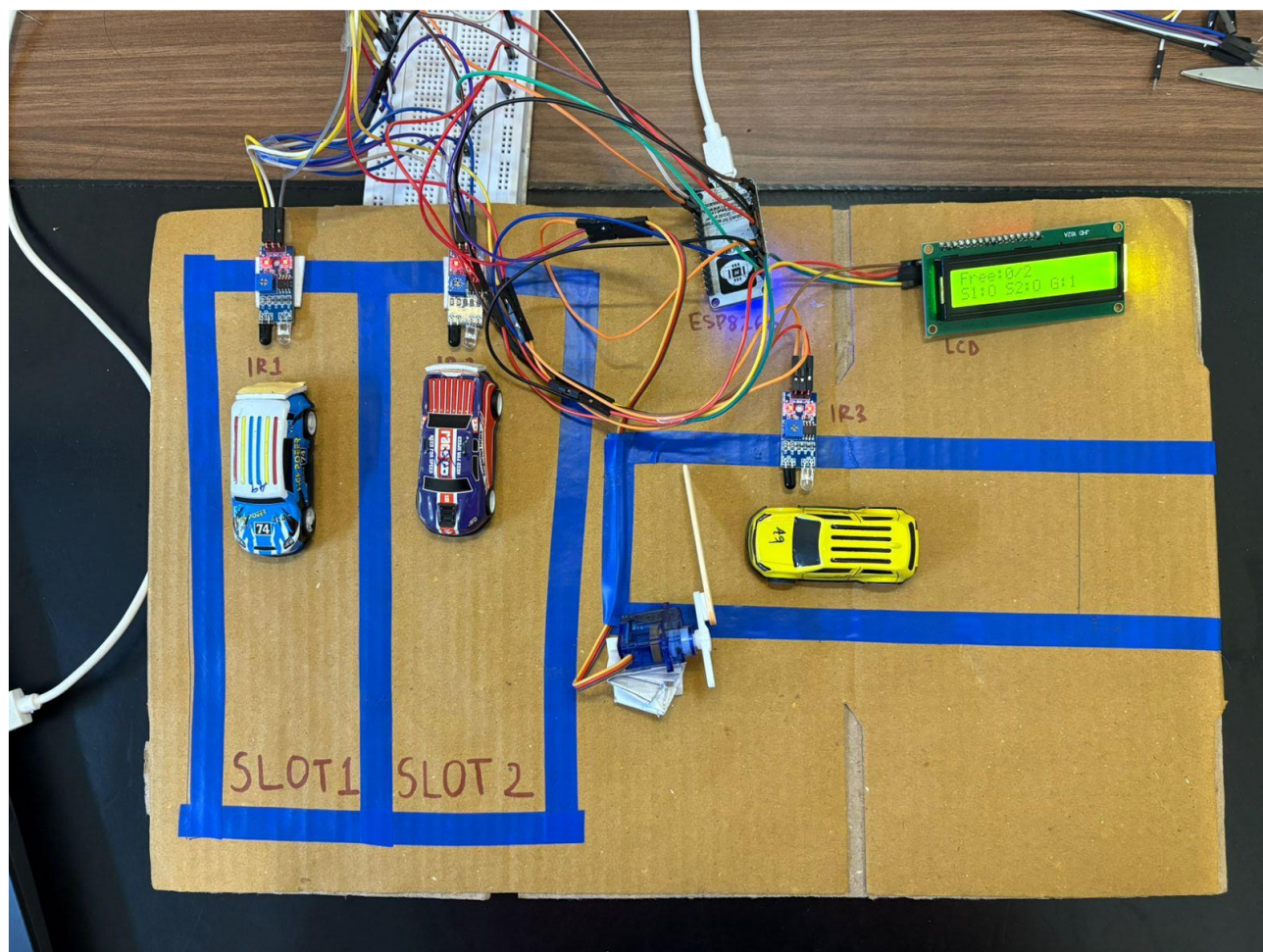
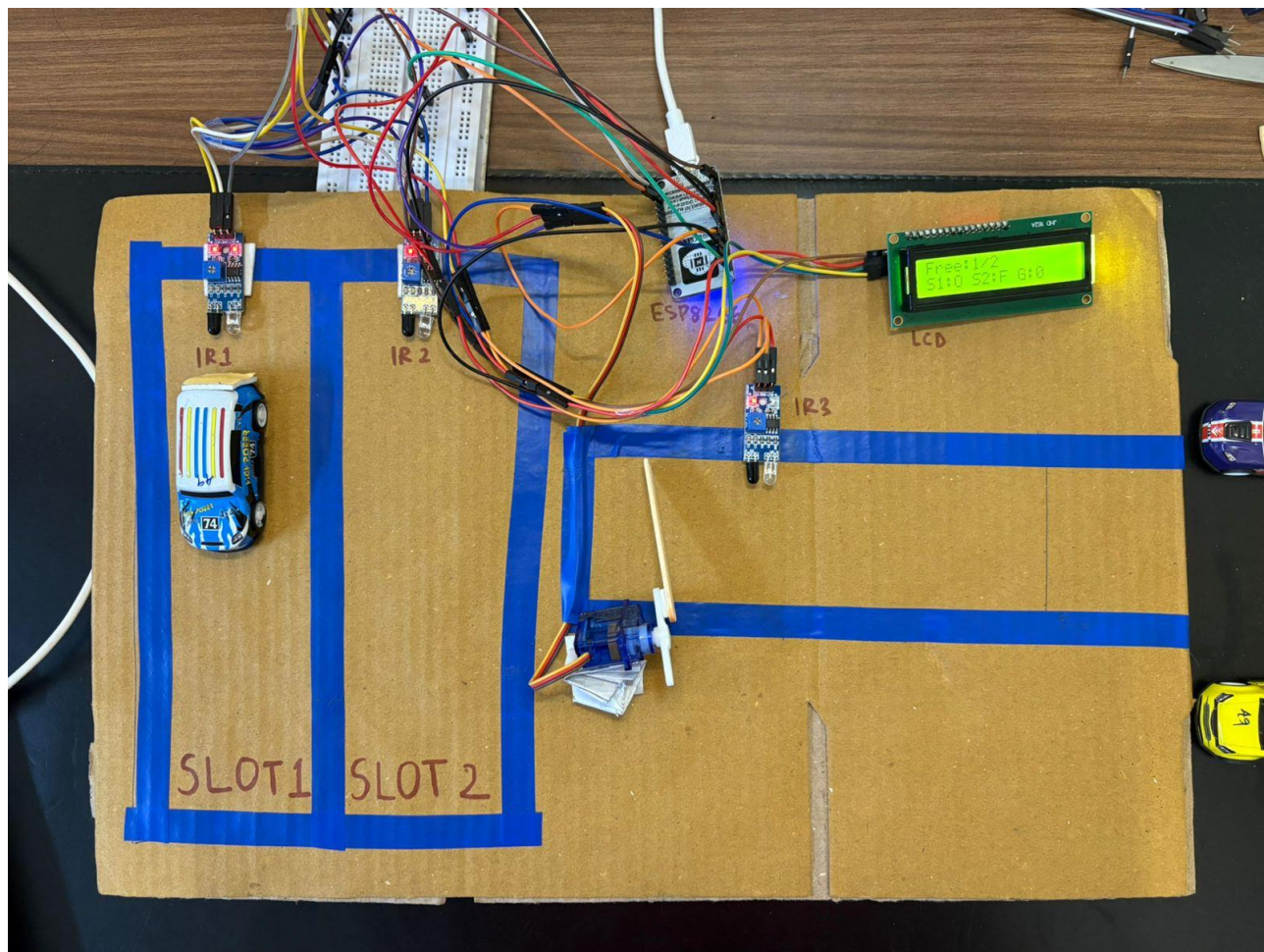
```
S1: OCC   S2: FREE
```

```
Free: 0 / 2  
PARKING FULL
```

When a vehicle shows up at the gate and the lot is full, the display can be optionally updated to indicate a full lot, e.g.:

3.8 Implementation photos



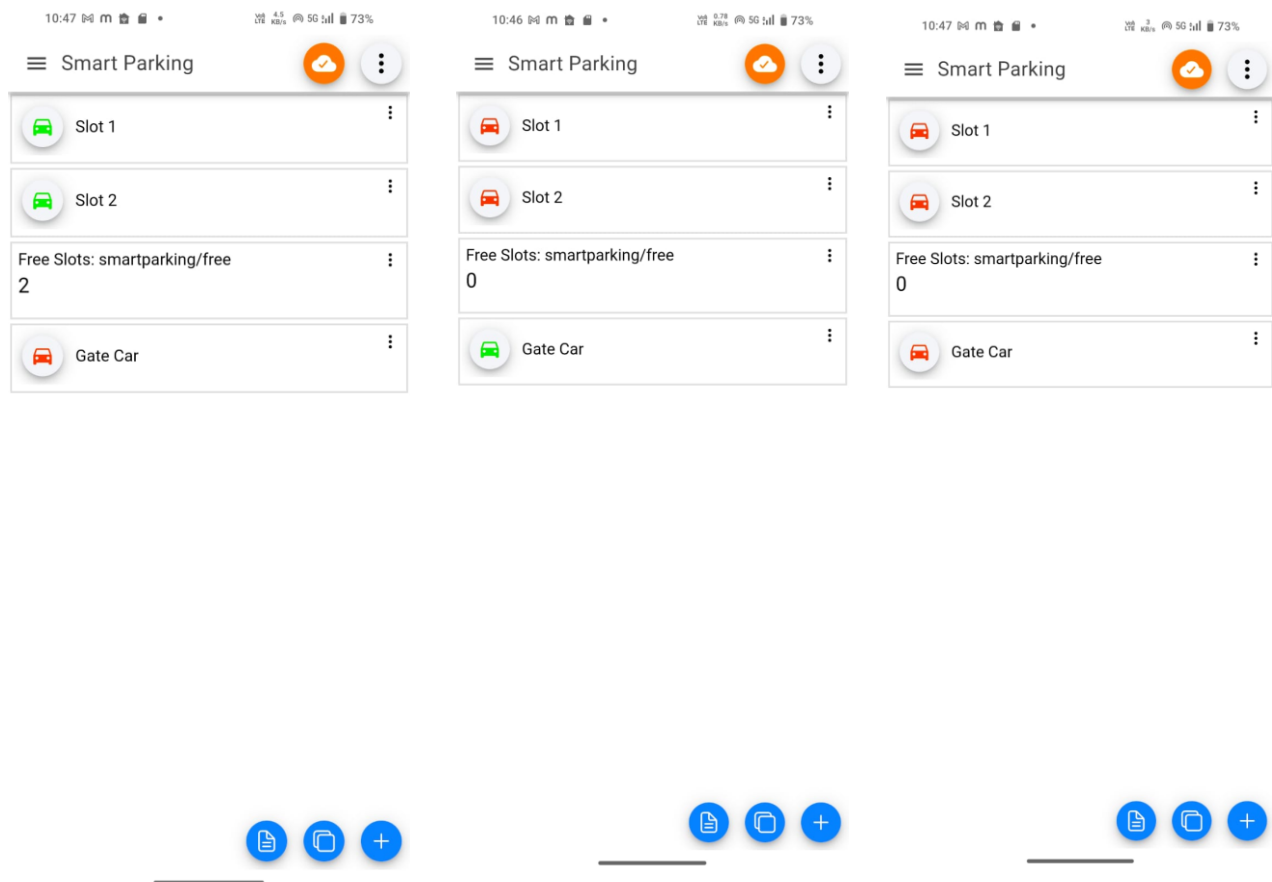


```
Car_Parking_System_Using_MQTT_Protocol | Arduino IDE 2.3.8
File Edit Sketch Tools Help
NodeMCU 1.0 (ESP-12E...)
Car_Parking_System_Using_MQTT_Protocol.ino
1 #include <ESP8266WiFi.h>
2 #include <PubSubClient.h>
3 #include <Servo.h>
4 #include <Wire.h>
5 #include <LiquidCrystal_I2C.h>
6
7 // WiFi
8 const char* ssid = "rabbay";
9 const char* password = "rabbayagatator";
10
11 // MQTT
12 const char* mqtt_server = "broker.emqx.io";
13 const int mqtt_port = 1883;
14
15 // Pins
16 #define SL0T1_PIN D5
17 #define SL0T2_PIN D6
18 #define GATE_SERVO_PIN D7
19 #define SERVO_PIN D4
20
21 LiquidCrystal_I2C lcd(0x27, 16, 2);
22
23 WiFiClient espClient;
24 PubSubClient client(espClient);
25 Servo gateServo;
26
27 bool slot1, slot2, gatecar;
28 int freeslots;
29 unsigned long gateopenuntil = 0;
30 bool gateopen = false;
31
32 void connectWifi() {
33   WiFi.mode(WIFI_STA);
34   WiFi.begin(ssid, password);
35   while (WiFi.status() != WL_CONNECTED) {
36     delay(1000);
37     Serial.println("");
38   }
39   Serial.println();
40   Serial.println("WiFi Connected");
41 }
42
43 void connectMQTT() {
44   while (!client.connected()) {
45     String clientId = "NodeMCU_SmartParking_";
46     clientId += String(floor(random(10000, 100000)), HEX);
47     if (client.connect(clientId.c_str())) {
48       // Success
49     }
50   }
51 }
52
53 void setup() {
54   Serial.begin(115200);
55   pinMode(SL0T1_PIN, OUTPUT);
56   pinMode(SL0T2_PIN, OUTPUT);
57   pinMode(GATE_SERVO_PIN, OUTPUT);
58   pinMode(SERVO_PIN, OUTPUT);
59   gateServo.attach(SERVO_PIN);
60   gateServo.write(0);
61   gateopen = false;
62   gateopenuntil = 0;
63   Serial.println("Setup Complete");
64 }
65
66 void loop() {
67   // Check if gate is open
68   if (gateopen) {
69     // Gate is open, check if it's time to close it
70     if (millis() > gateopenuntil) {
71       // Time to close the gate
72       gateopen = false;
73       gateopenuntil = 0;
74       Serial.println("Gate Closed");
75     }
76   }
77   // Check if slot 1 is occupied
78   if (slot1) {
79     // Slot 1 is occupied, check if it's time to free it
80     if (millis() > 1000) {
81       // Time to free the slot
82       slot1 = false;
83       Serial.println("Slot 1 Free");
84     }
85   }
86   // Check if slot 2 is occupied
87   if (slot2) {
88     // Slot 2 is occupied, check if it's time to free it
89     if (millis() > 1000) {
90       // Time to free the slot
91       slot2 = false;
92       Serial.println("Slot 2 Free");
93     }
94   }
95   // Check if gate car is present
96   if (gatecar) {
97     // Gate car is present, check if it's time to open the gate
98     if (millis() > 1000) {
99       // Time to open the gate
100      gatecar = false;
101      gateopen = true;
102      gateopenuntil = millis() + 1000;
103      Serial.println("Gate Open");
104    }
105  }
106  // Update LCD display
107  lcd.setCursor(0, 0);
108  lcd.print("Free Slots: ");
109  lcd.print(freeslots);
110  lcd.setCursor(0, 1);
111  lcd.print("Gate Status: ");
112  if (gateopen) {
113    lcd.print("Open");
114  } else {
115    lcd.print("Closed");
116  }
117  lcd.setCursor(0, 2);
118  lcd.print("Gate Car: ");
119  if (gatecar) {
120    lcd.print("Present");
121  } else {
122    lcd.print("Absent");
123  }
124  lcd.setCursor(0, 3);
125  lcd.print("Slot 1: ");
126  if (slot1) {
127    lcd.print("Occupied");
128  } else {
129    lcd.print("Free");
130  }
131  lcd.setCursor(0, 4);
132  lcd.print("Slot 2: ");
133  if (slot2) {
134    lcd.print("Occupied");
135  } else {
136    lcd.print("Free");
137  }
138  delay(1000);
139 }
140
141 // Output
142 Serial Monitor
143
144 wrote 206120 bytes (212699 compressed) at 0.000000000 in 18.8 seconds (effective 123.8 kbit/s)...
145 Hash of data verified.
146
147 Leaving...
148 Hard resetting via RTS pin...
```

```
Car_Parking_System_Using_MQTT_Protocol | Arduino IDE 2.3.8
File Edit Sketch Tools Help
NodeMCU 1.0 (ESP-12E...)
Car_Parking_System_Using_MQTT_Protocol.ino
54 } else {
55   Serial.println("MQTT failed, retry");
56   Serial.println(client.state());
57   delay(1000);
58 }
59 }
60
61 void publishState() {
62   client.publish("smartparking/slot1", slot1 ? "1" : "0", true);
63   client.publish("smartparking/slot2", slot2 ? "1" : "0", true);
64   client.publish("smartparking/gatecar", gatecar ? "1" : "0", true);
65   String freestr = String(freeslots);
66   client.publish("smartparking/free", freestr.c_str(), true);
67   client.publish("smartparking/gate", gateopen ? "open" : "closed", true);
68 }
69
70 void updateLCD() {
71   lcd.setCursor(0, 0);
72   lcd.setCursor(0, 1);
73   lcd.setCursor(0, 2);
74   lcd.setCursor(0, 3);
75   lcd.setCursor(0, 4);
76   lcd.setCursor(0, 5);
77   lcd.setCursor(0, 6);
78   lcd.setCursor(0, 7);
79   lcd.setCursor(0, 8);
80   lcd.setCursor(0, 9);
81   lcd.setCursor(0, 10);
82   lcd.setCursor(0, 11);
83   lcd.setCursor(0, 12);
84   lcd.setCursor(0, 13);
85   lcd.setCursor(0, 14);
86   lcd.setCursor(0, 15);
87 }
88
89 void setup() {
90   Serial.begin(115200);
91   pinMode(SL0T1_PIN, OUTPUT);
92   pinMode(SL0T2_PIN, OUTPUT);
93   pinMode(GATE_SERVO_PIN, OUTPUT);
94   pinMode(SERVO_PIN, OUTPUT);
95   gateServo.attach(SERVO_PIN);
96   gateServo.write(0);
97   gateopen = false;
98   gateopenuntil = 0;
99 }
100
101 void loop() {
102   // Check if gate is open
103   if (gateopen) {
104     // Gate is open, check if it's time to close it
105     if (millis() > gateopenuntil) {
106       // Time to close the gate
107       gateopen = false;
108       gateopenuntil = 0;
109       Serial.println("Gate Closed");
110     }
111   }
112   // Check if slot 1 is occupied
113   if (slot1) {
114     // Slot 1 is occupied, check if it's time to free it
115     if (millis() > 1000) {
116       // Time to free the slot
117       slot1 = false;
118       Serial.println("Slot 1 Free");
119     }
120   }
121   // Check if slot 2 is occupied
122   if (slot2) {
123     // Slot 2 is occupied, check if it's time to free it
124     if (millis() > 1000) {
125       // Time to free the slot
126       slot2 = false;
127       Serial.println("Slot 2 Free");
128     }
129   }
130   // Check if gate car is present
131   if (gatecar) {
132     // Gate car is present, check if it's time to open the gate
133     if (millis() > 1000) {
134       // Time to open the gate
135       gatecar = false;
136       gateopen = true;
137       gateopenuntil = millis() + 1000;
138       Serial.println("Gate Open");
139     }
140   }
141   // Update LCD display
142   updateLCD();
143   delay(1000);
144 }
145
146 // Output
147 Serial Monitor
148
149 wrote 206120 bytes (212699 compressed) at 0.000000000 in 18.8 seconds (effective 123.8 kbit/s)...
150 Hash of data verified.
151
152 Leaving...
153 Hard resetting via RTS pin...
```

```
Car_Parking_System_Using_MQTT_Protocol | Arduino IDE 2.3.8
File Edit Sketch Tools Help
NodeMCU 1.0 (ESP-12...)
Car_Parking_System_Using_MQTT_Protocol.ino
195 pinMode(GATE_SENSOR_PIN, INPUT);
196
197 gateServo.attach(SERVO_PIN);
198 closeGate();
199
200 // Wire.begin(D2, D1);
201 I2C.begin();
202 I2C.begin(0);
203 I2C.end();
204 I2C.begin("Smart Parking");
205
206 connectWiFi();
207 client.setServer(mqtt_server, mqtt_port);
208 connectMQTT();
209 delay(1500);
210
211 }
212
213 void loop() {
214   if (client.isConnected() != WL_CONNECTED) {
215     connectWiFi();
216   }
217   if (!client.isConnected()) {
218     connectMQTT();
219   }
220   client.loop();
221
222   // IR logic: Low = object detected
223   slot1 = (digitalRead(slot1_PIN) == LOW);
224   slot2 = (digitalRead(slot2_PIN) == LOW);
225   gateCar = (digitalRead(GATE_SENSOR_PIN) == LOW);
226
227   int occupiedSlots = (slot1 ? 1 : 0) + (slot2 ? 1 : 0);
228   freeSlots = 2 - occupiedSlots;
229
230   // Gate opens only if car is at gate and at least one slot is free
231   if ((gateopen && gateCar && freeSlots > 0) {
232     openGateServo(180);
233   }
234   if ((gateopen && millis() > gateopenuntil) {
235     closeGate();
236   }
237   publishState();
238   updateUI();
239   delay(1000);
240 }
241 }
242 }
243 }
244 }
245 }
246 }
247 }
248 }
249 }
250 }
251 }
252 }
253 }
254 }
255 }
256 }
257 }
258 }
259 }
260 }
261 }
262 }
263 }
264 }
265 }
266 }
267 }
268 }
269 }
270 }
271 }
272 }
273 }
274 }
275 }
276 }
277 }
278 }
279 }
280 }
281 }
282 }
283 }
284 }
285 }
286 }
287 }
288 }
289 }
290 }
291 }
292 }
293 }
294 }
295 }
296 }
297 }
298 }
299 }
300 }
301 }
302 }
303 }
304 }
305 }
306 }
307 }
308 }
309 }
310 }
311 }
312 }
313 }
314 }
315 }
316 }
317 }
318 }
319 }
320 }
321 }
322 }
323 }
324 }
325 }
326 }
327 }
328 }
329 }
330 }
331 }
332 }
333 }
334 }
335 }
336 }
337 }
338 }
339 }
340 }
341 }
342 }
343 }
344 }
345 }
346 }
347 }
348 }
349 }
350 }
351 }
352 }
353 }
354 }
355 }
356 }
357 }
358 }
359 }
360 }
361 }
362 }
363 }
364 }
365 }
366 }
367 }
368 }
369 }
370 }
371 }
372 }
373 }
374 }
375 }
376 }
377 }
378 }
379 }
380 }
381 }
382 }
383 }
384 }
385 }
386 }
387 }
388 }
389 }
390 }
391 }
392 }
393 }
394 }
395 }
396 }
397 }
398 }
399 }
400 }
401 }
402 }
403 }
404 }
405 }
406 }
407 }
408 }
409 }
410 }
411 }
412 }
413 }
414 }
415 }
416 }
417 }
418 }
419 }
420 }
421 }
422 }
423 }
424 }
425 }
426 }
427 }
428 }
429 }
430 }
431 }
432 }
433 }
434 }
435 }
436 }
437 }
438 }
439 }
440 }
441 }
442 }
443 }
444 }
445 }
446 }
447 }
448 }
449 }
450 }
451 }
452 }
453 }
454 }
455 }
456 }
457 }
458 }
459 }
460 }
461 }
462 }
463 }
464 }
465 }
466 }
467 }
468 }
469 }
470 }
471 }
472 }
473 }
474 }
475 }
476 }
477 }
478 }
479 }
480 }
481 }
482 }
483 }
484 }
485 }
486 }
487 }
488 }
489 }
490 }
491 }
492 }
493 }
494 }
495 }
496 }
497 }
498 }
499 }
500 }
501 }
502 }
503 }
504 }
505 }
506 }
507 }
508 }
509 }
510 }
511 }
512 }
513 }
514 }
515 }
516 }
517 }
518 }
519 }
520 }
521 }
522 }
523 }
524 }
525 }
526 }
527 }
528 }
529 }
530 }
531 }
532 }
533 }
534 }
535 }
536 }
537 }
538 }
539 }
540 }
541 }
542 }
543 }
544 }
545 }
546 }
547 }
548 }
549 }
550 }
551 }
552 }
553 }
554 }
555 }
556 }
557 }
558 }
559 }
560 }
561 }
562 }
563 }
564 }
565 }
566 }
567 }
568 }
569 }
570 }
571 }
572 }
573 }
574 }
575 }
576 }
577 }
578 }
579 }
580 }
581 }
582 }
583 }
584 }
585 }
586 }
587 }
588 }
589 }
590 }
591 }
592 }
593 }
594 }
595 }
596 }
597 }
598 }
599 }
600 }
601 }
602 }
603 }
604 }
605 }
606 }
607 }
608 }
609 }
610 }
611 }
612 }
613 }
614 }
615 }
616 }
617 }
618 }
619 }
620 }
621 }
622 }
623 }
624 }
625 }
626 }
627 }
628 }
629 }
630 }
631 }
632 }
633 }
634 }
635 }
636 }
637 }
638 }
639 }
640 }
641 }
642 }
643 }
644 }
645 }
646 }
647 }
648 }
649 }
650 }
651 }
652 }
653 }
654 }
655 }
656 }
657 }
658 }
659 }
660 }
661 }
662 }
663 }
664 }
665 }
666 }
667 }
668 }
669 }
670 }
671 }
672 }
673 }
674 }
675 }
676 }
677 }
678 }
679 }
680 }
681 }
682 }
683 }
684 }
685 }
686 }
687 }
688 }
689 }
690 }
691 }
692 }
693 }
694 }
695 }
696 }
697 }
698 }
699 }
700 }
701 }
702 }
703 }
704 }
705 }
706 }
707 }
708 }
709 }
710 }
711 }
712 }
713 }
714 }
715 }
716 }
717 }
718 }
719 }
720 }
721 }
722 }
723 }
724 }
725 }
726 }
727 }
728 }
729 }
730 }
731 }
732 }
733 }
734 }
735 }
736 }
737 }
738 }
739 }
740 }
741 }
742 }
743 }
744 }
745 }
746 }
747 }
748 }
749 }
750 }
751 }
752 }
753 }
754 }
755 }
756 }
757 }
758 }
759 }
760 }
761 }
762 }
763 }
764 }
765 }
766 }
767 }
768 }
769 }
770 }
771 }
772 }
773 }
774 }
775 }
776 }
777 }
778 }
779 }
780 }
781 }
782 }
783 }
784 }
785 }
786 }
787 }
788 }
789 }
790 }
791 }
792 }
793 }
794 }
795 }
796 }
797 }
798 }
799 }
800 }
801 }
802 }
803 }
804 }
805 }
806 }
807 }
808 }
809 }
810 }
811 }
812 }
813 }
814 }
815 }
816 }
817 }
818 }
819 }
820 }
821 }
822 }
823 }
824 }
825 }
826 }
827 }
828 }
829 }
830 }
831 }
832 }
833 }
834 }
835 }
836 }
837 }
838 }
839 }
840 }
841 }
842 }
843 }
844 }
845 }
846 }
847 }
848 }
849 }
850 }
851 }
852 }
853 }
854 }
855 }
856 }
857 }
858 }
859 }
860 }
861 }
862 }
863 }
864 }
865 }
866 }
867 }
868 }
869 }
870 }
871 }
872 }
873 }
874 }
875 }
876 }
877 }
878 }
879 }
880 }
881 }
882 }
883 }
884 }
885 }
886 }
887 }
888 }
889 }
890 }
891 }
892 }
893 }
894 }
895 }
896 }
897 }
898 }
899 }
900 }
901 }
902 }
903 }
904 }
905 }
906 }
907 }
908 }
909 }
910 }
911 }
912 }
913 }
914 }
915 }
916 }
917 }
918 }
919 }
920 }
921 }
922 }
923 }
924 }
925 }
926 }
927 }
928 }
929 }
930 }
931 }
932 }
933 }
934 }
935 }
936 }
937 }
938 }
939 }
940 }
941 }
942 }
943 }
944 }
945 }
946 }
947 }
948 }
949 }
950 }
951 }
952 }
953 }
954 }
955 }
956 }
957 }
958 }
959 }
960 }
961 }
962 }
963 }
964 }
965 }
966 }
967 }
968 }
969 }
970 }
971 }
972 }
973 }
974 }
975 }
976 }
977 }
978 }
979 }
980 }
981 }
982 }
983 }
984 }
985 }
986 }
987 }
988 }
989 }
990 }
991 }
992 }
993 }
994 }
995 }
996 }
997 }
998 }
999 }
1000 }
```

Output Serial Monitor
write 200120 bytes (212609 compressed) at 800000000 in 11.8 seconds (effective 123.8 kbit/s)...
Hash of data verified.
Leaving...
Hard resetting via RTS pin...



Implementation video link:

<https://youtu.be/zB74RBQmxC0>

3.9 Key Design Considerations

1. Two-Condition Gate Logic: This gate is only opened when the conditions are both true at the same time - a vehicle is detected at the gate sensor AND there is a slot free. Assessment of either of the two conditions would lead to the wrong behaviour of the gate.
2. IR Sensor Logic Check: Polarity of IR sensor output (active-LOW or active-HIGH) can differ among module variants. The firmware needs to be adjusted to the particular modules they are trying to test their output when setting up.
3. Gate Sensor Location: The gate IR sensor should be placed in such a way that it is able to sense a waiting vehicle in front of the gate mechanism and not behind it. This may lead to the opening of a gate to a car that has already partially entered.
4. Servo Timing and Gate Dwell: The firmware should have a correct dwell time after the gate has been opened to allow the vehicle to pass completely before the gate is opened once more. Such timing is to be changed depending on physical installation.
5. ESP8266 Voltage Tolerance: The GPIO pins are powered at 3.3V logic. When connecting 5V signals without a level shifter, there is the risk of damaging the microcontroller.
6. Servo Power Supply: Servo motors are large current consumers when in action. It is suggested that an independent 5V power supply should be used to prevent voltage drops on the supply rail of the NodeMCU.
7. Common Ground Rule: The entire components including any external power supply of the servo should have a common ground with the NodeMCU to have proper signal reference levels.

4. Conclusion

4.1 Summary of Findings

This project manages to illustrate the design and implementation of an IoT based smart parking system, in which a well-thought-out three-sensor architecture is used: two slot-monitoring IR sensors that continuously determine occupancy and one gate-entry IR sensor that determines the presence of a vehicle. This difference is a key design enhancement versus more basic systems, in that it guarantees the gate is only triggered by a real, actionable event - a vehicle at the gate when there is space.

The NodeMCU ESP8266 has been found to be a useful and small platform to interface sensors, provide conditional actuation control, local display, and Wi-Fi Internet connectivity, and MQTT in one low-cost device. The two condition two-way gate logic - vehicle at gate AND slot free - was used in a reliable way and makes sure that no unnecessary gate operations are performed, which is significant in terms of gate durability and processing of parking flows.

I2C LCD display provided a feasible aspect of local usability, with on-site staff or approaching drivers being able to read the parking status without the use of mobile apps. The remote monitoring using MQTT through the internet of things MQTT Panel established

that the status of the slot and gate could be relayed over Wi-Fi to a mobile interface to remotely administer the device.

Some important technical lessons were learned in implementation: the location of the sensors in the case of the gate IR sensor is sensitive, sensor output polarity must be carefully calibrated, the timing of the gate dwell is important in order to give vehicles time to pass safely, and consistent power supply is important to ensure the servo functions reliably.

4.2 Future Enhancements

- Adding more slot IR sensors to the system to scale it to accommodate more parking slots and accordingly change the structure of MQTT topics and the LCD layout.
- Substituting the IR gate sensor with an ultrasonic sensor that is more long-range and fewer false alarms due to ambient light interference.
- Adding a cloud database like Firebase or InfluxDB to store historical occupancy and gate event data to make analytics.
- Introduction of a web-based parking dashboard that can be used to manage multi-zone parking via any browser.
- Installing a mobile application to book and reserve slots in advance and the gate would automatically open on booking.
- Adding vehicle counting logic to triple slot sensor information with gate entry/exit occurrences to detect faults.
- Using solar energy to power the system and have a rechargeable battery backup in case of a remote or outdoors installation.

4.3 Final Remarks

The IoT-Based Smart Parking System, discussed in this report, is a practical, scalable and inexpensive prototype that illustrates how embedded systems and wireless IoT communication can be integrated to solve an issue in the real-world setting of a city. The incorporation of a special gate sensor to activate conditional gate control, instead of using timers or human intervention, is an intelligent and realistic solution to automating parking. This system is fully based on open-source libraries, highly accessible hardware, and common communication protocols, which offer a robust and re-producible basis of creating commercial-grade smart parking systems to add to the smart city infrastructure.

References

1. Espressif Systems. ESP8266 Technical Reference. Espressif Systems, 2023.
2. MQTT.org. MQTT Version 3.1.1 Protocol Specification. OASIS Standard, 2014.
3. Arduino. Arduino Reference Documentation. Arduino LLC, 2024.
<https://www.arduino.cc/reference/en/>
4. PubSubClient Library. Nick O'Leary, MQTT Client for Arduino. GitHub, 2023.
<https://github.com/knolleary/pubsubclient>
5. LiquidCrystal_I2C Library. Frank de Brabander. GitHub, 2023.
https://github.com/johnrickman/LiquidCrystal_I2C
6. Gubbi, J. et al. Internet of Things (IoT): A vision, architectural elements, and future directions. Future Generation Computer Systems, 29(7), 1645-1660, 2013.
7. Khanna, A. & Anand, R. IoT based Smart Parking System. Proceedings of the International Conference on Internet of Things and Applications (IOTA), IEEE, 2016.